# DEPARTMENT OF
# COMPUTER SCIENCE & ENGINEERING

## Experiment No. 9

**Student Name:** Arshpreet Singh
**UID:** 23BCS10502
**Branch:** BE CSE
**Section/Group:** KRG 1B
**Semester:** 6th
**Subject Code:** 23CSH-314
**Subject Name:** System Design
**Date of Performance:** 16/04/2026

## 1. Aim:

To design a highly scalable ticket booking platform that allows users to discover and purchase tickets for entertainment events, ensuring high availability for event discovery and strict consistency for seat reservations.

## 2. Objective

- Understand the architecture of a massive-scale ticketing and reservation system.
- Identify functional requirements such as event discovery, real-time seat mapping, and booking execution.
- Identify non-functional requirements including handling 100 Million DAU, search availability, and transaction consistency.
- Analyze the decoupling of payment workflows and status updates using event-driven stream processing (Kafka).
- Design REST APIs for event metadata, seat layouts, and secure transaction processing.

## 3. Procedure

- Studied real-world ticket booking architectures and concurrent transaction handling.
- Identified core entities such as Users, Events, Venues, Shows, Seats, and Bookings.
- Analyzed TTL-based distributed locking mechanisms to temporarily hold seats and prevent double-booking.
- Collected functional and non-functional requirements for event discovery and high-demand ticket sales.
- Designed APIs for search filtering, comprehensive event display, and booking execution.
- Evaluated CQRS-like patterns by separating read-heavy search operations (via Elasticsearch and CDC pipelines) from write-heavy primary databases.
- Analyzed asynchronous message brokers (Apache Kafka) for handling third-party payment callbacks, seat status updates, and user notifications.

## 4. System Requirements:

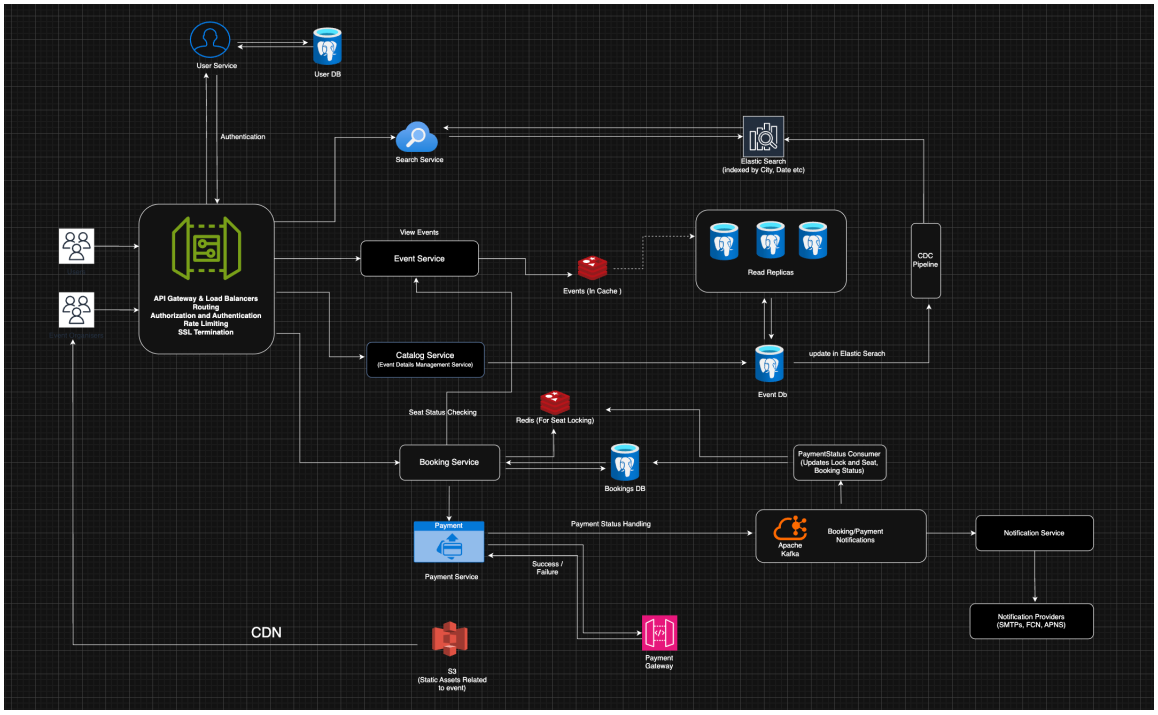
### A. Functional Requirements

- Users can discover events using filters such as Event Title, City/Location, and Date.
- Users can view comprehensive details for a specific event, including descriptions and metadata (cast, duration, language).
- Users can view real-time seat availability (available vs. occupied) for a specific show.
- The system allows users to select specific seats and complete a transaction to book a ticket.

### B. Non-Functional Requirements

- The system must be designed to scale and support 100 Million Daily Active Users (DAU).
- The platform must maintain High Availability for searching and viewing events (prioritizing always-online browsing, even with slightly stale data).
- The system must maintain High Consistency during the booking phase so that two people can never book the exact same seat concurrently.

## 5. High Level Design (HLD):



## 6. Learning Outcomes (What I Have Learnt)

- Designed a highly scalable, strictly consistent ticket booking architecture.
- Identified critical functional and non-functional requirements for high-demand event ticketing.
- Designed REST APIs and decoupled asynchronous processing pipelines for payments and notifications.
- Understood the application of Elasticsearch for fast read-heavy queries, TTL-based database locks for concurrency control, and Kafka for robust event-driven workflows.